

The LLM Grounding Playbook

[The LLM Grounding Playbook](#)
[Introduction](#)
[Getting Started](#)
[Why Grounding?](#)
[What is Grounding?](#)
[How to Ground LLMs](#)
[AI](#)
[RDF](#)
[RDFS/OWL](#)
[SPARQL](#)
[SQL](#)
[SHACL](#)
[demo](#)
[Palantir Foundry](#)
[AIP / MCP](#)
[LLM](#)
[A ·](#)
[B ·](#)
[C ·](#)

The LLM Grounding Playbook

中文副标题 [LLM](#)
Palantir [版本](#) v1.0 · 2026-07
完整教程 [作者手记](#) Palantir
Foundry / AIP [目标读者](#)
[配套代码](#) Python + 12
Foundry

前言

作者建议 [遇到错误](#) [04-zero-install-demo/README.md](#)

目录

[Introduction](#)
[Getting Started](#)

[知识图谱入门](#)
[AI 知识图谱](#)
[RDF](#)
[RDFS/OWL](#)
[SPARQL](#)
[SHACL](#)
[demo](#)
[Palantir Foundry](#)
[AIP / MCP](#)
[LLM](#)
[ABAC / CDC /](#)
[A ·](#)
[B ·](#)
[C ·](#)

第 0 章 · 写在前面

这本书解决什么问题？

[AI](#)
[LLM](#) 本体

哲学 2400 年的“分类学”
 知识表示 60 年的演变
 4 个核心规范 — RDF / RDFS / OWL / SHACL / SPARQL
 5 分钟跑通第一个 **demo**
 Docker
 Python
 7 个真实业务案例 — HR/ CRM
 Palantir Foundry 风格企业平台 — Object/Action/Function + ABAC + CDC
 AIP / MCP 集成 LLM

谁应该读？

☒ AI 应用开发者
☒ 软件架构师
☒ 数据治理工程师
☒ 技术负责人

怎么读？

30 分钟版
 4 小时版
 3 周系统学

第 1 章 · 哲学源流

世界怎么分类?

1.1 亚里士多德（前 384 - 前 322） · 范畴与定义

对今天的影响 RDFS `rdfs:subClassOf`

1.2 莱布尼茨（1646 - 1716） · 通用语言

对今天的影响 URI + RDF

1.3 康德（1724 - 1804） · 范畴与判断

对今天的影响 AI

1.4 弗雷格（1848 - 1925） · 谓词逻辑

对今天的影响 OWL Description Logic

1.5 1990s · 哲学进入工程

年	事件	影响
	"ontology"	
	Gruber	最常被引用的定义
	Studer	
	Noy & McGuinness	工程师入门必读

1.6 Gruber 1993 的经典定义（人话版）

本体 = 把大家头脑里“我们怎么理解这个领域”这件事，写成机器能读的文档。

“大家”“怎么理解”
“写成”OWL
“机器能读”= AI

第 2 章 · 知识表示

2.1 60 年时间线

1968 Quillian
1975 Minsky
1985 KL-ONE
1998 Tim Berners-Lee
2004 RDF W3C
2004 OWL 1.0
2009 OWL 2
2011 SPARQL 1.0
2017 SHACL
2024 Anthropic MCP Agent
2025 Palantir Foundry
2026 LLM + Ontology

2.2 为什么 2025 年又火本体?

LLM
GraphRAG
MCP Agent
本体的角色变了 AI

第3章 · AI时代为什么需要本体

3.1 LLM 的三大“病”

3.1.1 幻觉 (Hallucination)

症状????????????????根因????????????????????
 ???????本体怎么治? LLM ?????????先去本体查?????????
 ?????????????

3.1.2 数据孤岛

症状?????MySQL????PostgreSQL????Oracle??本体怎么
治?????schema????????????????

3.1.3 黑盒

症状??
 ???

3.2 一张决策表

场景	需要本体?	为什么
□□□□	□□□□	□□□□□□□
□□□□□□□□	□□□□	□□□□□□□□□□□□□□
□□□□	□□□□	□□□□□□□□□□
□□□□	□□□	□□□□
□□□□□□□	□□□	□□□□□□□□schema
□□Agent□□	□□□□	Agent□□□□□□

3.3 决策树

[illegible]

第 4 章 · RDF：所有事实的最小单元

4.1 一句话

RDF = 一切皆”主-谓-宾”三元组 (Triple)

4.2 第一个 RDF

```
@prefix ex: <http://example.com/> .  
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
```

```
ex:Neo a ex:Hacker ;  
    ex:realName "Thomas Anderson" ;  
    ex:loves ex:Trinity .
```

```
ex:Trinity a ex:Hacker ;  
    ex:loves ex:Neo .
```

```
[[[[[Neo [[[[[Hacker[[[[[[[[[[Thomas Anderson[[[[[[[[Trinity[[
```

4.3 5 个最常用模式

```
[[[[[ex:Alice a ex:Person[[[[[[[[[[  
[[[[[ex:Alice ex:age 30[[[[[[[[[[  
[[[[[ex:Alice ex:knows ex:Bob[[[[[[[[[[  
[[[[[ex:Alice rdfs:label "Alice"@zh[[[[[[[[[[  
[[[[[ex:Alice ex:birthday "1995-01-01"^^xsd:date[[[[[[[[[[
```

4.4 5 分钟上手 (Python)

```
from rdflib import Graph, Namespace, Literal  
from rdflib.namespace import RDF
```

```
g = Graph()  
EX = Namespace("http://example.com/")
```

```
g.add((EX.Alice, RDF.type, EX.Person))  
g.add((EX.Alice, EX.age, Literal([[[[[[[[[  
g.add((EX.Alice, EX.knows, EX.Bob))
```

```
print(g.serialize(format="turtle"))
```

输出

```
@prefix ns1: <http://example.com/> .
```

```
ns1:Alice a ns1:Person ;
```

```
ns1:age 30 ;
ns1:knows ns1:Bob .
```

第 5 章 · RDFS/OWL : 让机器”懂”

5.1 RDFS : 类层级

```
ex:Hacker rdfs:subClassOf ex:Person .
ex:Tom a ex:Hacker .
[[[?x][?y][?z][?w][?v][?u][?t][?s][?r]]Tom [[?x][?y]Person
```

5.2 OWL : 5 个类构造器

```
[[[?x][?y][?z][?w]]
ex:EmployedStudent owl:equivalentClass [
    owl:intersectionOf (ex:Employee ex:Student)
[[[?x][?y][?z]
```

```
[[[?x][?y][?z][?w]]
ex:FamilyMember owl:equivalentClass [
    owl:unionOf (ex:Parent ex:Child ex:Spouse)
[[[?x][?y][?z]
```

```
[[[?x][?y][?z][?w][?v][?u]]
ex:Person rdfs:subClassOf [
    owl:onProperty ex:hasChild ;
    owl:allValuesFrom ex:Human
[[[?x][?y][?z]
```

5.3 OWL : 5 个属性特征

```
ex:ancestorOf a owl:TransitiveProperty .
ex:marriedTo a owl:SymmetricProperty .
ex:Male owl:disjointWith ex:Female .
ex:hasBirthday a owl:FunctionalProperty .
ex:hasParent owl:inverseOf ex:hasChild .
```

第 6 章 · SPARQL : 本体的 SQL

6.1 5 个最常用模式

```
[[[?x][?y][?z][?w][?v][?u]]
SELECT ?x WHERE { ?x a ex:Class }
```

```
[[[?x][?y][?z][?w][?v][?u]]
```

```
SELECT ?x WHERE { ?x ex:age ?a . FILTER (?a > 18) }
```

```
SELECT ?x ?y WHERE { ?x a ex:Class . OPTIONAL { ?x ex:y ?y } }
```

```
SELECT ?x WHERE { { ?x a ex:A } UNION { ?x a ex:B } }
```

```
SELECT ?x WHERE { ex:Alice ex:knows+ ?x }
```

6.2 5 分钟上手：查 Wikidata

```
from SPARQLWrapper import SPARQLWrapper, JSON
```

```
sparql = SPARQLWrapper("https://query.wikidata.org/sparql")
```

```
sparql.setReturnFormat(JSON)
```

```
sparql.setQuery("""
```

```
PREFIX wd: <http://www.wikidata.org/entity/>
```

```
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
```

```
SELECT ?film ?filmLabel WHERE {
```

```
  ?film wdt:P57 wd:Q25191 .      # ?film
```

```
  SERVICE wikibase:label { bd:serviceParam wikibase:language
```

```
    "en" . }
```

```
} LIMIT 5
```

```
""")
```

```
for r in sparql.query().convert()["results"]["bindings"]:
```

```
    print(r["filmLabel"]["value"])
```

输出

Inception

The Dark Knight

Interstellar

"""

第 7 章 · SHACL：业务规则守门员

7.1 业务问题

```
OWL 规则用于描述业务规则
```

7.2 第一个 SHACL Shape

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
```

```
@prefix ex: <http://example.com/> .
```

```
ex:OrderShape
  a sh:NodeShape ;
  sh:targetClass ex:Order ;
  sh:property [
    sh:path ex:customer ;
    sh:minCount 1 ;
    sh:class ex:Customer ;
  ]
```

7.3 10 个最常用约束

约束	含义
sh:minCount N	N 个或更多
sh:maxCount N	最多 N 个
sh:datatype xsd:int	整数
sh:minInclusive N	大于等于 N
sh:pattern "..."	匹配正则表达式
sh:in (...)	在集合中
sh:class ex:X	实例化 ex:X 类

第 8 章 · 5 分钟跑通第一个 demo

8.1 装环境

```
pip install rdflib pyshacl SPARQLWrapper
```

8.2 跑第一个 demo

```
cd ~/Documents/projects/ai/ontology/04-zero-install-demo
python3 examples/1-first-rdf.py
```

输出

```
Graph with 2 nodes and 2 edges:
  http://example.com/Alice -- ...knows --> ...Bob
  http://example.com/Alice -- ...type --> ...Person
```

第 9 章 · 7 个真实业务案例

9.1 案例对照

案例	路径	适合
HR 招聘	05-real-cases/hr/	HR/????
客服	05-real-cases/customer-service/	????
财务	05-real-cases/finance/	????????
商品	05-real-cases/product/	????????
医疗	05-real-cases/medical/	???IT/??
物流	05-real-cases/logistics/	????????
CRM	05-real-cases/crm/	?????VP

9.2 每个案例的结构

case-name/	
├─ README.md	← ??????????
├─ case-query.py	← ??????
└─ data/	
├─ case-ont.ttl	← ??
├─ case-shapes.ttl	← SHACL ??
└─ case-data.ttl	← ?????

9.3 跑 HR 案例

```
cd ~/Documents/projects/ai/ontology/05-real-cases/hr
python3 hr-query.py
```

输出?????

?????✓????

Q1: ??????????
??????????

Q2: Offer ??????
?????????????Python ???

Q4: ?????
 Interviewing: 2 ?
 OfferSent: 1 ?
?????



第 10 章 · Palantir Foundry 风格平台

10.1 架构对齐

Palantir 概念 我们的实现

Object Type	owl:Class[?][?][?][?][?][?][?]
Link Type	ObjectProperty[?][?][?owl:inverseOf[?][?][?][?][?][?]
Property	DatatypeProperty[?][?][?][?][?][?][?]
Action Type	[?][?][?][?][?Python [?][?][?][?][?][?][?]
Function	Python [?][?][?][?][?][?][?]
SHACL	Object Validation (7 [?][?Shape)
Pipeline Builder	R2RML [?][?][?][?]
Workshop	HTML [?][?][?]
AIP	MCP Server (7 [?][?])

10.2 4 个工程模块

```
cd ~/Documents/projects/ai/ontology/05-real-cases/foundry-  
platform
```

```
[?][?][?][?][?][?][?][?][?][?section  
python3 run_platform.py
```

```
# 2. AIP / MCP Server  
python3 mcp_server.py
```

```
# 3. R2RML [?][?][?SQL → RDF[?  
python3 r2rml_materialize.py
```

```
# 4. HTML [?][?][?  
python3 build_dashboard.py
```

10.3 端到端工程流程

```
ERP/MySQL  
  ↓ r2rml_materialize.py  
RDF Graph  
  ↓ SHACL [?][?]  
[?][?][?][?][?][?][?][?][?][?]  
  ├── run_platform.py  
  ├── mcp_server.py → Claude Desktop  
  └── build_dashboard.py → dashboard.html
```

第 11 章 · AIP / MCP : 让 LLM 调本体

11.1 MCP Server

```
python3 mcp_server.py --mcp
```

7 个 MCP 工具

工具	作用
query_orders	
get_order_detail	
dashboard_kpi	KPI
reconcile_orders	
approve_order	
reserve_inventory	
assign_customer	

11.2 接入 Claude Desktop

```
~/Library/Application Support/Claude/claude_desktop_config.json
```

```
{
  "mcpServers": {
    "platform": {
      "command": "python3"
      "args": ["/path/to/mcp_server.py" "--mcp"]
    }
  }
}
```

11.3 对话示例

```
“查华为的所有订单” Claude query_orders(customer_name="")
```

第 12 章 · 工程化进阶

12.1 ABAC 权限

```
abac_security.py
```

```
Public / Internal / Confidential / Restricted
[ ]
[ ]
```


Level 3 (实战)

??????
???"VIP?"
?????
?????
?????

附录 A · 资源与工具速查

A.1 W3C 规范

规范	链接
RDF 1.1	https://www.w3.org/TR/rdf11-concepts/
RDFS	https://www.w3.org/TR/rdf-schema/
SPARQL 1.1	https://www.w3.org/TR/sparql11-query/
OWL 2	https://www.w3.org/TR/owl2-overview/
SHACL	https://www.w3.org/TR/shacl/
R2RML	https://www.w3.org/TR/r2rml/

A.2 Python 工具

库	装
rdflib	<code>pip install rdflib</code>
pyshacl	<code>pip install pyshacl</code>
owlready2	<code>pip install owlready2</code>
SPARQLWrapper	<code>pip install SPARQLWrapper</code>
mcp	<code>pip install mcp</code>

A.3 三元组库

工具	适合
Apache Jena + Fuseki	?????
GraphDB	??
Stardog	????
Oxigraph	???

A.4 编辑器

工具	网址
Protégé	https://protege.stanford.edu/

工具	网址
TopBraid Composer	https://www.topquadrant.com/
WebVOWL	http://vowl.visualdataweb.org/

A.5 公共数据源

源	数据
Wikidata	https://www.wikidata.org/
DBpedia	https://dbpedia.org/
schema.org	https://schema.org/

附录 B · 常见陷阱与最佳实践

B.1 七大常见陷阱

- 过度建模
- 命名不一致
- 混淆 TBox 和 ABox
- 谓词方向混乱
- 闭世界假设搞反
- 时间维度没处理
- URI 不稳定

B.2 十大最佳实践

- schema.org / FOAF
-
-
- Noy & McGuinness “”
-
- label
-
-
-
-

附录 C · 术语表

术语	一句话
本体 (Ontology)	
类 (Class)	

术语	一句话
实例 (Instance)	???????
属性 (Property)	?????
三元组 (Triple)	RDF ?????
TBox	???????????
ABox	?????????
RDF	??????
OWL	Web ?????
SHACL	??????
SPARQL	RDF ?????
ABAC	???????????
CDC	??????
AIP	AI Platform? Palantir?
MCP	Model Context Protocol
GraphRAG	RAG + ?????

写在最后

????????????????????

????????? RDF + OWL ?????? AI ??????

????????? AI ??

????????????????????? RDFS subClassOf
 ?????????????????????? URI
 ?????????????????????? Ontology
 ?????????????????????? OWL Description Logic
 Gruber 1993 → "?????????????????
 2004 W3C → RDF ?????
 2017 → SHACL
 2024-11 → MCP ??
 2025-2026 → LLM + Ontology ??
 ?????????????????????

下一步?

?????????
 ?????????? MCP
 ?????????? Claude Desktop
 ??????????
 ??????????

最后一句话?

AI 没有本体 = 会说人话的算命先生 AI 有了本体 = 看过病历的医生

?? ??Hermes Agent ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ??